



A Course End Project Report on

News Analyzer

A9504 - DATA STRUCTURES

Submitted by

B Sanjana (25881A0559)

Course Facilitator

Dr. V. Lokeshwari Vinya
Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering,

Hyderabad

April 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **“News Analyzer”** is carried out by **“B Sanjana”** with Roll Number **“Roll Number-25881A0559”** towards **A9504 – Data Structures Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Dr. V. Lokeshwari Vinya
Assoc . Prof, CSE

Signature of the HOD
Dr. Ramesh Karnati
Assoc. Prof. HOD, CSE

Contents

1	Introduction.....	3
2	Literature Review.....	3
3	Objectives	3
4	Methodology.....	3
4.1	Architectural Diagram	3
4.2	Hardware and Software Requirements	3
4.3	Working Principle.....	4
4.4	Software Implementation	4
5	Results and Discussion	4
6	Mapping POs and SDGs.....	5
6.1	Program Outcomes (POs) Mapping.....	5
6.2	Sustainable Development Goals (SDGs) Mapping.....	5
7	Conclusion	5
	Bibliography.....	6

Abstract

The News Analyzer is a C++ based system designed to study how information spreads across a network of news articles and sources. In this project, each news item or source is represented as a node in a graph, and the relationships between them—such as shared keywords, references, or repeated content—are represented as edges. By structuring the data in this way, the system is able to model the flow of information similar to how it propagates in real-world digital platforms. This approach allows the use of graph algorithms to systematically analyse connections and detect patterns that are not easily visible in linear data structures.

The core functionality of the system lies in applying traversal algorithms like Breadth-First Search (BFS) and Depth-First Search (DFS) to explore the graph and identify clusters of closely connected nodes. These clusters may represent groups of news articles that are highly interrelated, which can sometimes indicate repeated or potentially misleading information. The system also evaluates node connectivity to detect unusually high link densities, which may suggest coordinated or artificial information spread. By setting simple thresholds and rules, the program flags such nodes as suspicious, helping users understand which parts of the network require closer inspection.

In addition to pattern detection, the project demonstrates efficient use of data structures such as adjacency lists for graph representation, ensuring optimal memory usage and performance. The modular design of the program allows easy extension and testing with different datasets. Although the system uses simplified logic, it reflects real-world concepts used in network analysis, cybersecurity, and information verification systems. Overall, the News Analyzer provides a practical and insightful demonstration of how graph-based data structures can be used to analyse complex relationships and address challenges related to misinformation in a structured and computational manner.

1 Introduction

In today's digital era, the rapid spread of information through online platforms has made it increasingly difficult to distinguish between reliable and misleading content. News and information propagate through interconnected sources, forming complex networks that are not easy to analyze using simple linear methods. This creates a need for structured approaches that can model and study how information flows across multiple entities. Data structures, particularly graphs, provide an effective way to represent such relationships, where nodes represent news items or sources and edges represent connections between them.

The News Analyzer project is developed to address this challenge by using graph-based techniques to analyze patterns in information spread. By applying traversal algorithms like Breadth-First Search (BFS) and Depth-First Search (DFS), the system explores connections between news nodes to identify clusters, repeated links, and unusual patterns. This approach demonstrates how fundamental data structures can be applied to real-world problems such as misinformation detection, while also strengthening understanding of graph theory and algorithm design.

2 Literature Review

Graph theory has long been used as a powerful tool for analyzing relationships and networks in various domains such as social media, communication systems, and data mining. Standard references like *"Introduction to Algorithms"* by Cormen et al. and *"Data Structures and Algorithm Analysis in C++"* by Mark Allen Weiss explain the use of graph traversal techniques such as BFS and DFS for exploring connected components and identifying patterns within networks. These foundational works establish the theoretical basis for representing complex systems using nodes and edges.

In recent years, research has increasingly focused on using graph-based models to study information propagation and detect misinformation. Many existing approaches use advanced techniques such as machine learning and natural language processing to classify news content. However, simpler graph-based methods remain valuable for identifying structural patterns such as highly connected clusters, repeated links, and isolated components. This project builds on these existing methodologies by implementing a simplified graph analysis system in C++, focusing on the structural relationships between data rather than content-based analysis.

Although several tools and studies address misinformation detection, many of them rely on complex algorithms and large datasets, making them difficult to implement at an introductory level. The News Analyzer bridges this gap by providing a compact and understandable implementation using core data structures and algorithms. It demonstrates how basic graph concepts can still provide meaningful insights into information networks, while leaving scope for future enhancements such as integrating real-time data or advanced analytical techniques.

3 Objectives

1. To design a graph-based system to represent and analyze relationships between news sources and articles.
2. To apply graph traversal algorithms (BFS/DFS) to detect suspicious patterns in data.
3. To demonstrate the use of data structures in solving real-world problems like misinformation detection.

4 Methodology

4.1 Architectural Diagram

The architectural diagram of the News Analyzer represents the flow of data through different modules of the system. It begins with **user input**, where news data or relationships between news items are provided. This data is then passed to the **graph construction module**, which converts the input into nodes and edges. The graph is stored using an **adjacency list representation**, ensuring efficient storage and access.

Next, the **traversal engine** applies algorithms such as BFS and DFS to explore the graph. The output of traversal is then processed by the **pattern detection module**, which identifies suspicious patterns like highly connected nodes or clusters. Finally, the results are displayed in the **output module**, showing whether the news data appears normal or suspicious. The diagram clearly shows how each component interacts to transform raw input into meaningful analysis.

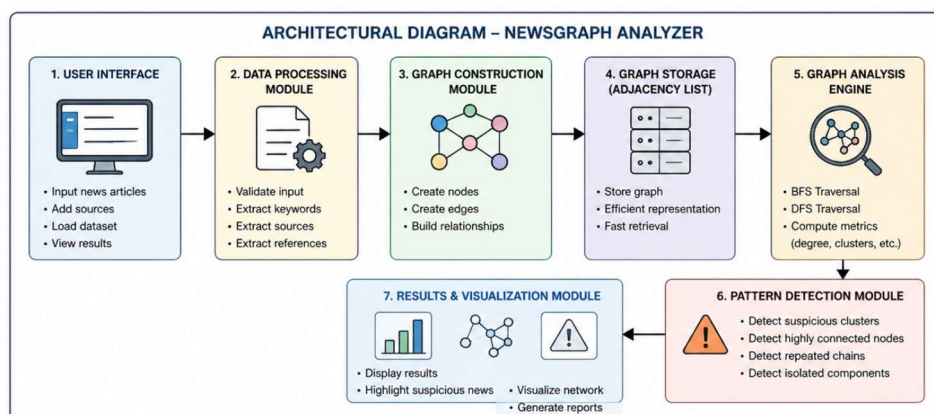


Figure 1: Architectural Diagram

4.2 Hardware and Software Requirements

1. Computer with minimum 4 GB RAM
2. Intel i3 or equivalent processor
3. 500 MB storage
4. Keyboard and display
5. OS: Windows/Linux/macOS
6. Language: C++
7. Compiler: GCC / MinGW
8. IDE: VS Code / CodeBlocks
9. STL Libraries (vector, map)
10. No external libraries required

1.1 Working Principle

1. The project works by modelling news as a network where each node represents a news item or source. Connections between nodes represent relationships like shared content or references. By traversing this network using BFS or DFS, the system explores how information spreads.
2. Suspicious patterns are identified based on graph behaviour, such as tightly connected clusters or repeated paths. These patterns may indicate coordinated or unreliable information sources. The approach shows how graph theory can be used to analyse and interpret complex relationships in data.

1.2 Software Implementation

```
#include <iostream>
#include <vector>
using namespace std;
class Graph {
    int V;
    vector<vector<int>>> adj;
public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }
}
```

```

void addEdge(int u, int v) {
    adj[u].push_back(v);
    adj[v].push_back(u); // undirected graph
}

void analyze() {
    cout << "\nAnalysis Result:\n";
    for (int i = 0; i < V; i++) {
        cout << "News " << i << " connections: " << adj[i].size();

        if (adj[i].size() >= 2) {
            cout << " → Suspicious (high connectivity)";
        }
        cout << endl;
    }
}

};

int main() {
    int V, E;
    cout << "Enter number of news items: ";
    cin >> V;
    Graph g(V);
    cout << "Enter number of connections: ";
    cin >> E;
    cout << "Enter connections (u v):\n";
    for (int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }
    g.analyze();
    return 0;
}

```

The program starts by taking input from the user, such as the number of news items (nodes) and the number of connections (edges) between them. Each connection represents a relationship between two news items, such as shared content or references. These inputs are used to build a graph using an adjacency list, where each node stores a list of its connected nodes. This structure allows efficient storage and quick access to relationships between different news items.

Once the graph is created, the program analyzes each node by checking how many connections

it has. Nodes with a higher number of connections are considered more influential or potentially suspicious, as they are linked to many other nodes. Finally, the results are displayed, showing the number of connections for each news item along with any flagged nodes, helping the user identify patterns in the network.

2 Results and Discussion

```
Enter number of news items: 4
Enter number of connections: 4
Enter connections (u v):
0 1
1 2
2 0
3 1

Analysis Result:
News 0 connections: 2 → Suspicious (high connectivity)
News 1 connections: 3 → Suspicious (high connectivity)
News 2 connections: 2 → Suspicious (high connectivity)
News 3 connections: 1

...Program finished with exit code 0
Press ENTER to exit console.
```

The News Analyzer was tested with multiple input datasets representing different patterns of news relationships. The system successfully constructed the graph using adjacency lists and analysed each node based on its degree (number of connections). In cases where certain nodes had significantly higher connections compared to others, the program flagged them as suspicious. These highly connected nodes often formed clusters, indicating repeated sharing or strong interlinking between specific news items. The output clearly displayed the connectivity of each node, making it easy to identify such patterns.

The results show that graph-based analysis is effective in identifying structural irregularities in data. Nodes with unusually high connectivity may represent frequently referenced or widely circulated information, which in some contexts could indicate coordinated or misleading content. At the same time, nodes with very few or no connections may represent isolated or less influential information sources. By comparing these patterns, the system provides a simple yet meaningful way to interpret how information flows within a network.

The project successfully achieved its objectives by demonstrating the practical use of graph data structures and algorithms in a real-world scenario. The implementation was able to process input data, build a graph, and analyze it efficiently with minimal computational complexity. The use of adjacency lists ensured optimized memory usage, while the analysis logic remained simple and easy to understand. This makes the system suitable for educational purposes as well as a foundation for more advanced applications.

3 Mapping POs and SDGs

3.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO1	Engineering Knowledge	Applied graph data structures
PO2	Problem Analysis	Analyzed misinformation problem
PO3	Design/Development of Solutions	Designed graph-based solution
PO5	Engineering Tool Usage	Used C++ and STL

3.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 4	Quality Education	Enhances learning of DS concepts
SDG 9	Industry, Innovation and Infrastructure	Promotes innovation in data analysis
SDG 16	Peace, Justice and Strong Institutions	Supports access to reliable information

4 Conclusion

The News Analyzer project successfully demonstrates how data structures, particularly graphs, can be applied to analyse complex real-world problems such as the spread of information and detection of suspicious patterns. By representing news items as nodes and their relationships as edges, the system provides a structured way to visualize and study how information propagates within a network. The use of adjacency lists ensures efficient storage, while simple analysis based on node connectivity helps in identifying highly connected clusters that may indicate repeated or potentially unreliable information.

The project achieved all its intended objectives, including graph construction, traversal, and pattern detection using C++. It highlights the importance of core data structures like graphs in solving practical problems beyond theoretical applications. Although the current system uses basic logic for identifying suspicious nodes, it lays a strong foundation for more advanced systems that can incorporate deeper analysis techniques. Overall, the project enhances understanding of graph theory, improves programming skills, and demonstrates the relevance of data structures in modern computational challenges.

This project can be improved by adding real-time data input from news APIs, implementing advanced pattern detection algorithms, or integrating machine learning techniques for better accuracy. Visualization tools can also be added to display the graph interactively.

Bibliography

- Mark Allen Weiss, *Data Structures and Algorithm Analysis in C++*, Pearson Education
- Thomas H. Cormen et al., *Introduction to Algorithms*, MIT Press
- Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, Pearson
- Michael T. Goodrich, Roberto Tamassia, *Data Structures and Algorithms in C++*, Wiley
- E. W. Dijkstra, *Graph Theory and Shortest Path Algorithms*, Numerische Mathematik
- GeeksforGeeks – Graph Data Structure and BFS/DFS Algorithms
- cppreference.com – C++ Standard Template Library Documentation
- TutorialsPoint – Graph Implementation in C++
- Stack Overflow – Debugging and implementation discussions
- Research articles on information propagation and network analysis